

Python Practicals with Answers

[Core Python](#) | [OOP](#) | [File Handling](#) | [Modules](#)

Variables, Data Types & Operators · Control Flow & Loops · Functions & Modules · Object-Oriented Programming · File Handling & Exception Handling

Practical 1: Variables, Data Types & Operators

Aim: Explore Python's data types, operators, type conversion and input/output.

Code:

```
# Data Types
x      = 10          # int
pi     = 3.14        # float
name   = "Yug"       # str
flag   = True        # bool
items  = [1,2,3]     # list
coords = (10, 20)    # tuple
info   = {"a": 1}     # dict
unique = {1, 2, 3}    # set

print(type(x), type(name)) # <class 'int'> <class 'str'>

# Operators
print(10 // 3) # 3 (floor division)
print(10 % 3)  # 1 (modulus)
print(2 ** 8)  # 256 (power)

# String operations
s = "Hello, Python!"
print(s.upper())      # HELLO, PYTHON!
print(s[0:5])         # Hello
print(s.replace("Python", "World")) # Hello, World!
print(len(s))         # 14

# Type conversion
print(int("42") + 8)  # 50
print(str(100) + " km") # "100 km"
print(float("3.14") * 2) # 6.28

# f-strings
age = 20
print(f"Name: {name}, Age: {age}, PI: {pi:.2f}")
# Output: Name: Yug, Age: 20, PI: 3.14

# Input
# name = input("Enter name: ") # reads from user
# print(f"Hello, {name}!")
```

Explanation:

Python is dynamically typed. **int**, **float**, **str**, **bool**: primitive types. **list**: mutable ordered. **tuple**: immutable ordered. **dict**: key-value pairs. **set**: unique unordered. **//**: floor division. ******: exponentiation. **f-strings**: `f'text {variable}'` for interpolation. **:.2f**: format float to 2 decimal places. **type()**: check data type.

Output: Output: , 3, 1, 256, HELLO PYTHON, Hello, 50, '100 km', Name: Yug, Age: 20, PI: 3.14

Practical 2: Control Flow & Loops

Aim: Use **if/elif/else**, **for/while** loops, **break/continue** and **list comprehensions**.

Code:

```
# if / elif / else
marks = 78
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}") # Grade: B

# for loop with range
for i in range(1, 6):
```

```

    print(i, end=" ")      # 1 2 3 4 5

# for loop over list
fruits = ["Mango", "Apple", "Banana"]
for i, fruit in enumerate(fruits):
    print(f"{i+1}. {fruit}")

# while loop
n = 1
while n <= 5:
    print(n, end=" ")
    n += 1                # 1 2 3 4 5

# break and continue
for i in range(10):
    if i == 3: continue   # skip 3
    if i == 7: break      # stop at 7
    print(i, end=" ")     # 0 1 2 4 5 6

# List Comprehension
squares = [x**2 for x in range(1,6)]    # [1,4,9,16,25]
evens    = [x for x in range(20) if x%2==0]
pairs    = [(x,y) for x in range(3) for y in range(3)]

print(squares)    # [1, 4, 9, 16, 25]
print(evens[:5])  # [0, 2, 4, 6, 8]

```

Explanation:

range(start, stop, step): generates numbers. **enumerate(list):** index+value pairs. **break:** exit loop early. **continue:** skip to next iteration. **List comprehension:** [expression for item in iterable if condition] — more Pythonic than for+append. **Ternary:** value_if_true if condition else value_if_false.

Output: Grade: B. Squares [1,4,9,16,25]. Evens [0,2,4,6,8]. Pairs all (0-2)x(0-2) combos. Loop skips 3, stops before 7.

Practical 3: Functions & Modules

Aim: Define and use functions, lambda, *args/**kwargs, and import modules.

Code:

```

# Basic function
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Yug"))          # Hello, Yug!
print(greet("Yug", "Hi"))    # Hi, Yug!

# *args and **kwargs
def show(*args, **kwargs):
    print("args: ", args)
    print("kwargs:", kwargs)

show(1, 2, 3, name="Yug", age=20)
# args:  (1, 2, 3)
# kwargs: {'name': 'Yug', 'age': 20}

# Lambda
square = lambda x: x ** 2
add     = lambda a, b: a + b
print(square(5), add(3,4))    # 25 7

# map, filter, sorted with lambda
nums = [3, 1, 4, 1, 5, 9, 2]
doubled = list(map(lambda x: x*2, nums))
positives = list(filter(lambda x: x>3, nums))
sorted_n = sorted(nums, reverse=True)
print(doubled)    # [6,2,8,2,10,18,4]
print(positives)  # [4, 5, 9]

# Importing modules

```

```
import math, random, datetime

print(math.sqrt(144))          # 12.0
print(math.ceil(4.3))          # 5
print(random.randint(1, 100))  # random int
print(datetime.date.today())   # today's date

# from ... import
from os.path import join, exists
print(join("folder", "file.txt")) # folder/file.txt
```

Explanation:

def fn(params): define function. **default params:** used when arg omitted. ***args:** captures extra positional args as tuple. ****kwargs:** captures keyword args as dict. **lambda x: expr:** anonymous single-expression function. **map(fn, list):** apply fn to each. **filter(fn, list):** keep where fn returns True. **import module** or **from module import name**.

Output: greet outputs 'Hello, Yugi'. args/kwargs correctly captured. doubled list multiplied by 2. math.sqrt(144)=12.0. Today's date printed.

Practical 4: Object-Oriented Programming

Aim: Create classes, objects, inheritance, and use special (dunder) methods.

Code:

```
# Class definition
class Student:
    college = "GTU"          # class variable

    def __init__(self, name, marks):
        self.name = name    # instance variables
        self.marks = marks

    def grade(self):
        if self.marks >= 90: return "A"
        elif self.marks >= 75: return "B"
        else: return "C"

    def __str__(self):        # called by print()
        return f"Student({self.name}, {self.marks}, {self.grade()})"

    def __repr__(self):
        return f"Student('{self.name}', {self.marks})"

# Create objects
s1 = Student("Yug", 85)
s2 = Student("Alice", 92)
print(s1)                    # Student(Yug, 85, B)
print(s2.grade())            # A
print(Student.college)       # GTU

# Inheritance
class PlacedStudent(Student):
    def __init__(self, name, marks, company):
        super().__init__(name, marks) # call parent __init__
        self.company = company

    def info(self):
        return f"{self.name} placed at {self.company}"

    def __str__(self):
        return f"Placed: {self.name} @ {self.company}"

p = PlacedStudent("Bob", 88, "TCS")
print(p)                     # Placed: Bob @ TCS
print(p.grade())             # B (inherited from Student)
print(p.info())              # Bob placed at TCS

# isinstance / issubclass
```

```
print(isinstance(p, Student))          # True
print(issubclass(PlacedStudent, Student)) # True
```

Explanation:

class Name: define class. **__init__(self,...):** constructor. **self:** instance reference. **class variable:** shared across all instances. **instance variable:** unique per object. **__str__:** print() representation. **__repr__:** debug representation. **super().__init__():** call parent constructor. **Inheritance:** child class gets all parent methods.

Output: s1 prints as 'Student(Yug, 85, B)'. PlacedStudent inherits grade() method, returns 'B'. isinstance confirms p is also a Student.

Practical 5: File Handling & Exception Handling

Aim: Read/write files and handle exceptions using try/except/finally.

Code:

```
import os

# Writing a file
with open("students.txt", "w") as f:
    f.write("Yug,85\n")
    f.write("Alice,92\n")
    f.write("Bob,78\n")

# Reading entire file
with open("students.txt", "r") as f:
    content = f.read()
    print(content)

# Reading line by line
with open("students.txt", "r") as f:
    for line in f:
        name, marks = line.strip().split(",")
        print(f"{name}: {marks}")

# Appending to file
with open("students.txt", "a") as f:
    f.write("Carol,95\n")

# Exception Handling
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except TypeError as e:
        print(f"Type Error: {e}")
        return None
    finally:
        print("safe_divide() finished")

print(safe_divide(10, 2))    # 5.0
print(safe_divide(10, 0))    # Error + None
print(safe_divide("a", 2))   # TypeError + None

# Custom Exception
class MarksError(Exception):
    pass

def validate_marks(m):
    if m < 0 or m > 100:
        raise MarksError(f"Invalid marks: {m}")
    return m

try:
    validate_marks(110)
```

```
except MarkError as e:
    print(e)                # Invalid marks: 110

# Cleanup
os.remove("students.txt")
```

Explanation:

with open(file, mode) as f: auto-closes file. Modes: 'r' read, 'w' write (overwrite), 'a' append, 'rb'/'wb' binary. **f.read():** full content. **f.readlines():** list of lines. **f.write(str):** write string. **try/except ExceptionType:** catch specific error. **finally:** always runs. **raise ExceptionClass(msg):** throw custom error. **class MyError(Exception):** custom exception.

Output: File created, read, each line parsed. `safe_divide(10,2)=5.0`; by zero prints error+None; type error caught. Custom exception prints 'Invalid marks: 110'.

-- End of Practicals --